

GPU-Accelerated Physics-Based Urban Flood Modelling

Using CuPy/CUDA and JAX/XLA

A Project Report submitted by

Sanjeev Kumawat

M.Sc. Student, Indian Institute of Technology Hyderabad

in partial fulfilment of the requirements for the
INSPIRE Summer Project

Under the Guidance of

Prof. Udit Bhatia

Associate Professor, Indian Institute of Gandhinagar



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

August 2026

Declaration

I declare that this report presents the computational work and evidence contained in the project archive identified in the reproducibility statement. Sources of theory and software are acknowledged in the references. Numerical verification is distinguished throughout from validation against observations; no unavailable measurements, simulations, or AI results have been invented.

Sanjeev Kumawat

M.Sc. student

Department of Physics

Indian Institute of Technology Hyderabad

Date: 4 August 2026

Place: Gandhinagar

Acknowledgement

I acknowledge the guidance of Prof. Udit Bhatia, the provision of the production domain and forcing data, and the use of the external GPU system on which the reported production runs were executed. The author remains responsible for the analysis and interpretation.

Sanjeev Kumawat

Abstract

This project investigates whether one physics-based urban flood model can be implemented consistently through two GPU programming systems: an explicit CuPy/CUDA implementation and a compiled JAX/XLA implementation. Both backends solve the two-dimensional shallow-water equations on the same unstructured triangular mesh and use the same terrain, spatially varying Manning resistance, hydrostatic-reconstruction HLL flux, wetting and drying rules, Horton infiltration, rainfall, recharge, drainage, surcharge, outfall, boundary and external-inflow data. The production experiment covers 297.966 km² of active area with 981,880 triangles, 1,490,015 surface edges, 118,768 drainage nodes and 139,798 drainage links. A 160.09 mm, 24-hour rainfall event was simulated in single precision on one NVIDIA RTX PRO 4000 Blackwell GPU without CPU fallback.

Cross-backend verification passed all eight approved numerical gates. The maximum hourly depth root-mean-square error was 0.000979348 m, the minimum hourly flood-extent critical success index was 0.998031, and maximum final depths differed by 7.63×10^{-6} m. Independent repeat runs showed RMSE values of 0.000416766 m for CuPy and 0.000406100 m for JAX. The runs were therefore numerically repeatable within the registered criteria but were not byte-for-byte identical. In a secondary 60-minute benchmark with three warm-up runs and ten measurements per backend, median solver runtimes were 17.267 s for CuPy and 18.213 s for JAX; JAX was 5.476% slower for this workload. This result is not substituted for the unfinished repeated 24-hour official benchmark. ANUGA was used only on small compatible controlled cases as an independent numerical reference. No observational flood data were available, and only one production event was available; consequently, physical validation, production AI training and operational forecasting claims are outside the evidence. The contribution is a tested, reproducible GPU numerical model and a scientifically qualified comparison of its two accelerator implementations.

Code availability: The implementation, experiment configurations, verification evidence and report sources are available at <https://github.com/arsanjeevk/gpu-urban-flood-cupy-jax>.

Keywords: shallow-water equations; urban flooding; GPU computing; CuPy; CUDA; JAX; XLA; finite volume method; numerical verification; drainage coupling.

Contents

Declaration	2
Acknowledgement	3
Abstract	4
Lists of Figures and Tables	7
Nomenclature	8
1 Introduction	9
1.1 Motivation	9
1.2 Scientific and computational background	9
1.3 Aim, objectives and research questions	10
1.4 Scope and contribution	10
1.5 Report organisation	10
2 Production Data and Computational Domain	11
2.1 Domain construction	11
2.2 Terrain and resistance	11
2.3 Rainfall, infiltration and boundaries	12
2.4 Drainage representation and provenance	12
2.5 Input preparation and quality assurance	13
2.6 Masks and hydraulic roles	13
3 Physical and Numerical Model	15
3.1 Governing equations	15
3.2 Finite-volume update and HLL flux	15
3.3 Hydrostatic reconstruction and terrain	15
3.4 Wetting, drying, resistance and velocity control	16
3.5 Rainfall and Horton infiltration	16
3.6 Surface–drainage coupling	16
3.7 Boundary conditions, timestep and update order	16
3.8 Discrete computation sequence	17
3.9 System mass ledger	17
4 GPU Implementation Architecture	18
4.1 One model, two accelerator paths	18
4.2 CuPy/CUDA implementation	18
4.3 JAX/XLA implementation	18
4.4 Role of PyTorch	18
4.5 Execution environment	19
4.6 Parallel work and device-resident data	19
4.7 Compilation, synchronization and output boundaries	19

5	Verification and Experimental Methodology	21
5.1	Evidence hierarchy	21
5.2	Production experiment	21
5.3	Parity metrics and acceptance gates	21
5.4	Repeatability protocol	21
5.5	Performance protocol	21
5.6	Controls against an invalid comparison	22
5.7	Software verification and package audit	22
6	Production Simulation Results	23
6.1	Surface and network response	23
6.2	Preserved state and diagnostic products	23
6.3	Wet extent and timing	24
6.4	Flood duration and velocity diagnostics	24
6.5	Mass-balance accounting	25
7	Numerical Parity and Repeatability	27
7.1	Cross-backend parity	27
7.2	Spatial differences	27
7.3	Within-backend repeatability	28
7.4	Interpretation of localized differences	28
8	GPU Performance Comparison	30
8.1	Completed secondary benchmark	30
8.2	Interpretation	30
8.3	Unfinished official 24-hour benchmark	31
9	ANUGA Numerical Reference	32
10	Limitations and Future Work	33
11	Conclusions	34
	Reproducibility and Evidence Statement	35
	References	36

Lists of Figures and Tables

Figures

2.1	Production surface mesh and drainage topology.	11
2.2	Terrain and hydraulic resistance fields used by both production backends.	12
2.3	Production rainfall hyetograph.	12
2.4	Production boundary representation.	14
6.1	Production surface-water results.	24
6.2	Thresholded flood extent and arrival-time products.	24
6.3	Duration and velocity products.	25
6.4	Surface and drainage storage histories.	26
7.1	Cross-backend parity through the event.	27
7.2	Spatial difference between JAX and CuPy depths.	28
8.1	Runtime distribution for the secondary benchmark.	30

Tables

2.1	Production-domain inventory obtained from the packaged topology summary.	11
2.2	Scientific input traceability.	13
3.1	Ordered production update and its conservative or diagnostic role.	17
4.1	Production GPU and software environment.	19
4.2	Backend correspondence in the production calculation.	20
5.1	Final automated-test outcome.	22
6.1	Principal 24-hour production outputs.	23
6.2	Selected preserved arrays from the CuPy production archive; the JAX archive has corresponding fields.	23
6.3	Detailed 24-hour mass-ledger terms.	26
7.1	Approved parity gates and observations.	27
7.2	Within-backend repeatability.	28
7.3	Selected hourly parity observations.	29
7.4	Complementary meaning of parity metrics.	29
8.1	Secondary repeated GPU benchmark.	30
9.1	Controlled-case ANUGA–JAX evidence.	32
11.1	Evidence retained with the completed project.	35

Nomenclature

Symbol	Meaning
h	Surface-water depth (m)
u, v	Depth-averaged velocities in the projected x and y directions (m s^{-1})
hu, hv	Conserved depth-integrated momentum components ($\text{m}^2 \text{s}^{-1}$)
z	Bed elevation (m, vertical datum undocumented)
$\mathbf{U}$	Conserved surface state vector $[h, hu, hv]^T$
$\mathbf{F}, \mathbf{G}$	Shallow-water flux vectors in the x and y directions
$\mathbf{S}$	Combined physical and coupling source vector
g	Gravitational acceleration, 9.80665 m s^{-2}
n	Manning resistance coefficient ($\text{s m}^{-1/3}$)
$r(t)$	Piecewise-constant rainfall intensity (m s^{-1} internally)
f_0, f_c, k	Initial capacity, final capacity and decay coefficient in the Horton model
A_i, L_e	Triangle area and edge length (m^2 and m)
Δt	Adaptive explicit solver timestep (s)
C_{CFL}	Courant–Friedrichs–Lewy coefficient
RMSE	Root-mean-square error
CSI	Critical success index for thresholded flood extent

Backend abbreviations. CUDA denotes NVIDIA’s GPU execution platform; CuPy denotes the NumPy-compatible CUDA array backend; JAX denotes the traced array-program backend; and XLA denotes the compiler used to lower the staged JAX computation. ANUGA is an external shallow-water reference and is not either production backend.

Chapter 1

Introduction

1.1 Motivation

Urban flood simulation involves rapid overland flow, heterogeneous terrain, rainfall losses, drainage systems, and boundary exchanges. At city scale, the computational mesh can contain millions of interacting elements, while explicit time integration requires repeated local flux and source-term evaluations. Consequently, computational efficiency is a major challenge for high-resolution flood simulation.

GPU acceleration is a promising approach because many of these local operations can be executed concurrently. However, computational speed alone is insufficient for a scientifically meaningful implementation. An accelerated solver must preserve the governing equations, input data, numerical decisions, and physical behaviour of the reference formulation. Therefore, performance evaluation must be accompanied by systematic numerical-parity checks.

This study focuses specifically on two GPU acceleration approaches: **CuPy** and **JAX**. CuPy provides NumPy-compatible GPU operations and supports custom CUDA kernels, whereas JAX uses just-in-time compilation through XLA. Both approaches are evaluated on the same complete production dataset for the Gurugram application. The objective is to compare the execution performance of the two GPU implementations while maintaining numerical consistency with the reference flood-model formulation.

1.2 Scientific and computational background

The shallow-water equations are a depth-integrated approximation to free-surface flow. They retain gravity-wave propagation and horizontal momentum transport while assuming that the horizontal length scale is large relative to water depth. In conservation form they admit discontinuities, moving wet-dry fronts and rapidly varying flow. A finite-volume method is therefore appropriate because it updates cell-integrated quantities through edge fluxes and can be constructed directly on an unstructured mesh [2]. Triangular cells are particularly useful in urban domains because they can follow irregular boundaries and local geometric detail without imposing a rectangular grid over the entire region.

Numerical treatment of topography is central to flood modelling. A naive discretisation may generate artificial motion even when the water surface should remain at rest. Hydrostatic reconstruction modifies the left and right depths seen by an interface flux and adds matched pressure corrections, providing a well-balanced treatment of bed steps while maintaining non-negative reconstructed water depth [4]. The HLL approximate Riemann solver estimates the fastest left- and right-travelling signals and constructs one conservative interface flux without explicitly resolving every characteristic wave [3]. Wetting and drying rules then prevent division by vanishing depth.

Rainfall is not identical to effective surface input because part of it may infiltrate or enter engineered drainage. The Horton relationship represents a capacity that decreases from an initial value toward a final value as wet time accumulates [5]. Surface resistance is represented through spatial Manning coefficients [6]. In this study, these quantities participate in the production update alongside recharge, inlet capture, pipe conveyance, surcharge, outfalls, and external boundary inflow.

The computational structure exposes large groups of independent edge and cell operations, but indexed accumulation and adaptive stepping complicate acceleration. CuPy permits direct control of device arrays and custom CUDA kernels [7]; JAX instead traces functional array code and lets XLA compile the staged computation [8]. These approaches therefore provide different computational mechanisms for executing the same underlying numerical formulation, which motivates

their evaluation under a common physical and computational framework.

1.3 Aim, objectives and research questions

The aim of this study is to evaluate the numerical correctness, repeatability, and measured execution performance of equivalent CuPy and JAX GPU implementations of a physics-based urban flood model.

The study has four objectives:

1. implement the production surface–drainage model using CuPy/CUDA and JAX/XLA;
2. ensure that both implementations follow the same physical and numerical formulation;
3. evaluate cross-backend numerical parity and within-backend repeatability; and
4. compare their execution performance under a repeated and synchronized GPU protocol.

These objectives lead to three research questions:

1. Does the JAX implementation reproduce the CuPy reference within the predefined numerical acceptance criteria?
2. Are repeated executions of each backend numerically stable within the predefined repeatability criteria, even when bitwise identity is not obtained?
3. What execution-time difference is observed between the two implementations for an identical repeated GPU workload?

1.4 Scope and contribution

The contribution of this study is numerical and computational rather than observational. The production model includes complete terrain and roughness fields, shallow-water dynamics, rainfall and Horton infiltration, recharge, dynamic drainage, surcharge, outfalls, external hydrograph forcing, inactive-cell walls, and outflow-only exterior boundaries. The study establishes software-test success, cross-backend numerical parity, within-backend repeatability, and measured GPU performance for the CuPy and JAX implementations.

The study does not establish physical validation of the predicted inundation because compatible observed depth, flood extent, discharge, or timing data were not available. The performance results are also specific to the tested hardware, software environment, production dataset, and benchmark protocol and should not be interpreted as a universal ranking of GPU frameworks.

1.5 Report organisation

Chapter 2 describes the production domain and forcing; Chapter 3 defines the physical and numerical model; Chapter 4 explains the two GPU implementation paths; and Chapter 5 describes the verification and experimental methodology. Chapters 6 to 8 present the production simulation, numerical parity and repeatability, and GPU performance results, respectively. Chapter 9 provides the limited ANUGA numerical reference, followed by the limitations and future work in Chapter 10 and the conclusions in Chapter 11.

Chapter 2

Production Data and Computational Domain

2.1 Domain construction

The production surface is an unstructured triangular mesh in EPSG:32643. It contains 499,901 vertices and 981,880 triangles connected by 1,490,015 surface edges. Of these triangles, 817,573 are active and 164,307 are inactive; inactive cells act as closed walls. The active area is 297.965823 km². There are 34,390 exterior edges with code 4, treated as outflow-only except where the historical external inflow is applied.

Table 2.1: Production-domain inventory obtained from the packaged topology summary.

Quantity	Count	Quantity	Count
Mesh vertices	499,901	Surface edges	1,490,015
All triangles	981,880	Active triangles	817,573
Inactive triangles	164,307	Exterior code-4 edges	34,390
Drainage nodes	118,768	Drainage links	139,798
Mapped inlets	61,317	Enabled outfalls	46
Recharge records	447	Hourly solver states	25

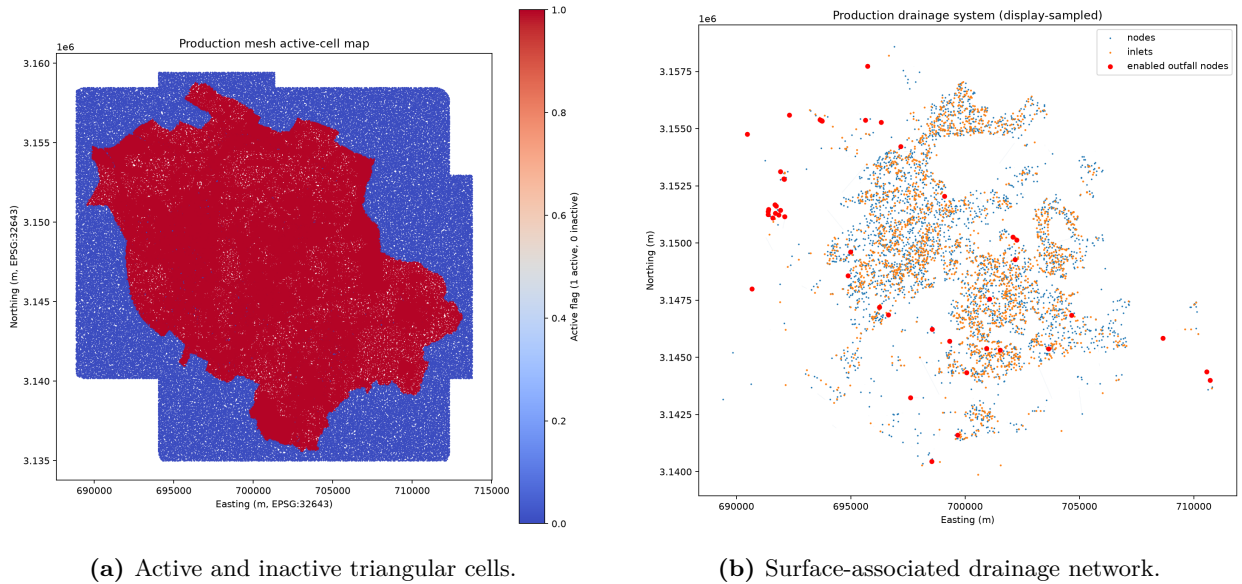
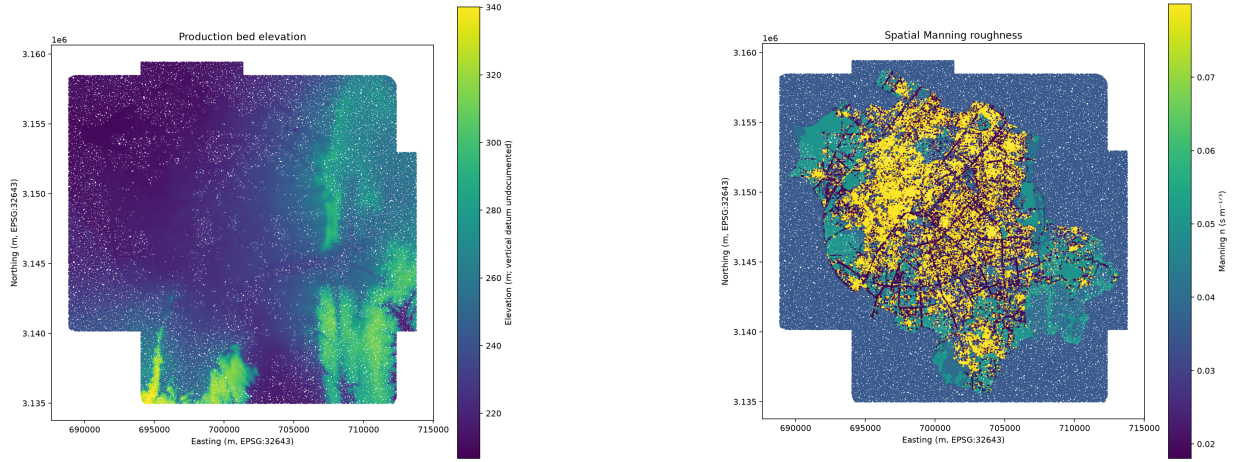


Figure 2.1: The production surface mesh and drainage topology. Figures are generated directly from the packaged production arrays.

2.2 Terrain and resistance

Centroid bed elevations are finite for all 981,880 triangles, so no runtime terrain filling is used in the production experiment. The source coordinate reference system is known, but the vertical datum is not documented. Elevation differences are used by hydrostatic reconstruction and hydraulic heads; absolute-elevation interpretation must therefore retain this datum limitation. Spatial Manning coefficients are also present for every triangle, ranging from 0.018 to 0.080 with mean 0.04613 in the production field. They are active numerical friction parameters, not metadata-only attributes.



(a) Centroid bed elevation.

(b) Spatial Manning coefficient.

Figure 2.2: Terrain and hydraulic resistance fields used by both production backends.

2.3 Rainfall, infiltration and boundaries

The forcing table contains 24 one-hour piecewise-constant intensities whose sum is 160.09 mm. Rainfall is spatially uniform because the cell multiplier is one, but losses are spatially heterogeneous through Horton fields. Initial Horton capacity f_0 ranges from 0.0128 to 7.2 mm h^{-1} ; final capacity f_c ranges from 0.004 to 2.25 mm h^{-1} ; the decay coefficient is 4 h^{-1} . The boundary subsystem includes an external hydrograph mapped to 150 receiver cells, with a reported peak total flow of $95.1714 \text{ m}^3 \text{ s}^{-1}$. These inputs are modelling data and priors; their presence does not by itself constitute observational validation of the simulated flood.

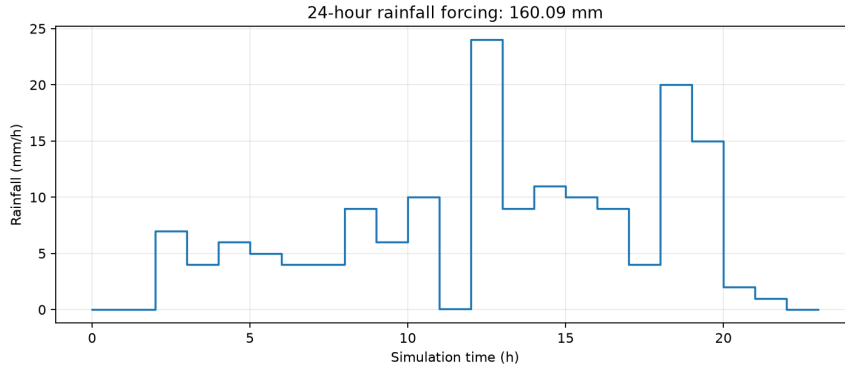


Figure 2.3: The 24-hour production rainfall hyetograph. The event total is 160.09 mm; intensities are converted from mm h^{-1} to m s^{-1} inside the model.

2.4 Drainage representation and provenance

Surface water may enter 61,317 mapped inlets, move through 139,798 links, leave through 46 enabled outfalls, return through surcharge and interact with 447 recharge records. The initial surface is dry, while drainage storage is initialized from the approved topology profile rather than assumed empty.

The package stores source manifests and SHA-256 hashes for the topology, Horton, recharge, boundary-inflow and rainfall assets. Required topology fields fail validation when absent; the production runner does not silently create missing scientific fields.

2.5 Input preparation and quality assurance

The production calculation did not operate directly on loosely related GIS layers. The source material was transformed into indexed arrays whose dimensions and connectivity agree with the computational mesh. Triangle centroids and vertex coordinates retain the projected EPSG:32643 reference, edge records identify their adjacent cells, and each active surface cell has a finite bed elevation and Manning value. Drainage links refer to valid node indices, while inlet and recharge records are explicitly mapped to receiving surface cells. This preprocessing is important because a physically meaningful coefficient attached to the wrong cell is still a numerically valid array and may otherwise pass simple shape checks.

The executed input path applies progressively stronger checks. First, required filenames and configuration keys must exist. Second, the loader checks array ranks, lengths, dtypes, finite values and index ranges. Third, topology summaries compare counts and active area with the expected production inventory. Finally, hashes preserve the identity of the scientific assets used by both backends. Missing required topology fields cause a failure instead of a default value being silently manufactured. The resulting readiness record marked the production topology as ready and reported no external-inflow snap warnings.

Table 2.2: Input role, execution representation and completed quality control.

Scientific input	Numerical representation	Completed check
Surface geometry	vertices, triangles, centroids, edges, areas and normals	Dimensions, connectivity, active mask and domain inventory recorded
Terrain and resistance	one elevation and Manning coefficient per triangle	All 981,880 elevations finite; full spatial roughness field retained
Rainfall and infiltration	hourly intensity series; spatial Horton arrays	Units converted at the model interface; 24-hour event and parameter ranges recorded
Drainage and recharge	node/link graph, inlet mapping and 447 recharge rows	Node and cell indices validated; six recharge sources explicitly remapped
Boundary forcing	code-4 exterior edges and 150 hydrograph receivers	Outflow semantics, receiver count, total hydrograph and spatial snap distances checked

2.6 Masks and hydraulic roles

The archived surface arrays distinguish active cells, buildings and roads. The active mask determines where the surface equations advance; it contains 817,573 true entries, or 83.266% of all triangles. Building and road masks cover 29.028% and 41.720% of the triangles, respectively. These masks provide spatial information used in the construction of hydraulic properties and domain status, while the production solver directly consumes the resulting active mask, elevations, roughness and Horton fields.

The boundary geometry is similarly explicit. Inactive neighbours are closed walls, whereas the 34,390 exterior code-4 edges permit outward flux. Historical external inflow is not introduced by reversing that edge rule; it is added as a separately accounted source to 150 mapped receiver cells. Keeping these two mechanisms separate makes outward boundary loss and prescribed inflow independently visible in the mass ledger.

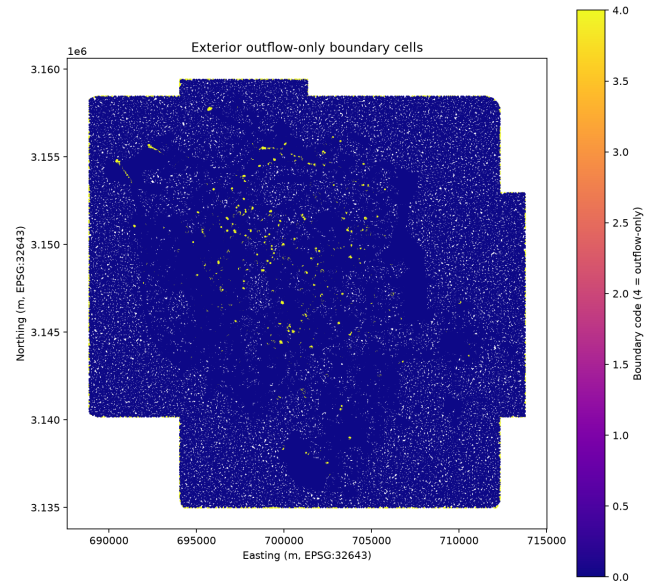


Figure 2.4: Exterior code-4 boundary locations in the production domain. Prescribed historical inflow is handled separately at mapped receiver cells. *Source: completed-project evidence package.*

Chapter 3

Physical and Numerical Model

3.1 Governing equations

The surface state in each cell is

$$\mathbf{U} = \begin{bmatrix} h & hu & hv \end{bmatrix}^T, \quad (3.1)$$

where h is water depth, u and v are depth-averaged velocities, and hu, hv are depth-integrated momenta. The conservative two-dimensional shallow-water system is written in the standard finite-volume conservation form [2]:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} + \frac{\partial \mathbf{G}(\mathbf{U})}{\partial y} = \mathbf{S}, \quad (3.2)$$

with

$$\mathbf{F} = \begin{bmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \\ huv \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} hv \\ huv \\ hv^2 + \frac{1}{2}gh^2 \end{bmatrix}. \quad (3.3)$$

The source vector contains rainfall, infiltration and exchange contributions to depth, together with bed and friction effects on momentum. Gravity is $g = 9.80665 \text{ m s}^{-2}$.

3.2 Finite-volume update and HLL flux

Integrating equation (3.2) over triangular cell i of area A_i gives the explicit update

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{A_i} \sum_{e \in \partial i} L_e \hat{\mathbf{F}}_{e,i} + \Delta t \mathbf{S}_i, \quad (3.4)$$

where L_e is edge length and $\hat{\mathbf{F}}$ is the numerical flux normal to the edge. For left and right wave-speed estimates s_L and s_R , the HLL middle flux is

$$\hat{\mathbf{F}}_{\text{HLL}} = \frac{s_R \mathbf{F}_L - s_L \mathbf{F}_R + s_L s_R (\mathbf{U}_R - \mathbf{U}_L)}{s_R - s_L}, \quad (3.5)$$

with the left or right physical flux selected when both waves travel in one direction [3]. This approximate Riemann solver is local to each edge and therefore maps well to parallel hardware.

3.3 Hydrostatic reconstruction and terrain

To combine fluxes with discontinuous bed elevation, the interface bed is $z^* = \max(z_L, z_R)$ and reconstructed depths are

$$h_L^* = \max(h_L + z_L - z^*, 0), \quad h_R^* = \max(h_R + z_R - z^*, 0). \quad (3.6)$$

The HLL flux uses these reconstructed depths, while paired pressure corrections $\frac{1}{2}g(h^2 - h^{*2})$ are applied to the two adjacent cells. This Audusse-style hydrostatic reconstruction supports non-negative depth and lake-at-rest balance [4]. Consequently, the DEM and spatially varying elevation are genuine inputs to the production update.

3.4 Wetting, drying, resistance and velocity control

A minimum depth of $h_{\min} = 10^{-5}$ m protects velocity division. Cells below this threshold have zeroed momenta. Manning resistance is applied semi-implicitly:

$$(hu, hv)^{n+1} = \frac{(hu, hv)^*}{1 + \Delta t gn^2 \|\mathbf{u}^*\| / \max(h, h_{\min})^{4/3}}, \quad (3.7)$$

where n is the spatial roughness coefficient; this resistance convention follows the standard Manning formulation [6]. A maximum allowed speed of 5 m s^{-1} is subsequently enforced by proportional momentum scaling. The depth is clipped non-negative after flux and exchange updates.

3.5 Rainfall and Horton infiltration

Rainfall intensity $r(t)$ is evaluated as a one-hour step function and converted to SI units. For cell multiplier m_i , gross rainfall depth added over a step is

$$\Delta h_{r,i} = r(t) m_i \Delta t. \quad (3.8)$$

Horton infiltration capacity evolves with accumulated wet time τ_i as

$$f_i(\tau_i) = f_{c,i} + (f_{0,i} - f_{c,i}) e^{-k_i \tau_i}, \quad (3.9)$$

and actual loss is limited by both $f_i \Delta t$ and available water [5]. This avoids removing more water than exists in a cell. Momentum is scaled consistently with remaining depth.

3.6 Surface–drainage coupling

For an inlet linking a surface cell to a drainage node, the implemented exchange follows an orifice-type head relation

$$Q_e = C_d A_e \sqrt{2g|H_s - H_n|} \operatorname{sgn}(H_s - H_n), \quad (3.10)$$

subject to inlet capacity and available-volume constraints. Pipe flow is advanced from head slope with Manning and minor-loss damping and is bounded by link capacity. Outfall discharge has the form

$$Q_o = C_o A_o \sqrt{2g|H_n - H_t|} \operatorname{sgn}(H_n - H_t), \quad (3.11)$$

then is limited to positive outflow, configured capacity and available node volume. Excess node storage returns to mapped surface cells as surcharge. Recharge captures water above a stop depth, stores it subject to capacity, and removes it at a specified recharge rate.

3.7 Boundary conditions, timestep and update order

Inactive neighbours generate reflective closed-wall states. Exterior code-4 edges allow outflow but block flux-directed inflow; the approved external hydrograph is added separately to mapped receiver cells. The stable step satisfies

$$\Delta t = \min \left[\Delta t_{\max}, C_{\text{CFL}} \min_i \frac{\ell_i}{\|\mathbf{u}_i\| + \sqrt{gh_i}} \right], \quad C_{\text{CFL}} = 0.85, \quad \Delta t_{\max} = 1 \text{ s}. \quad (3.12)$$

Each production step follows the same order in both backends: surface flux and friction; rainfall and Horton loss; velocity limiting; external inflow; inlet exchange; pipe and outfall flow; overflow return; recharge; and wet-time update. The output schedule produces 25 stored states over the 24-hour simulation.

3.8 Discrete computation sequence

The ordering of operations is part of the numerical model because nonlinear clipping, capacity limits and wet–dry decisions do not generally commute. Both backends therefore use the same staged sequence shown in table 3.1. Fluxes first move conserved quantities between adjacent triangles. Rainfall and external inflow add water; infiltration, outfalls and recharge loss remove or retain it; inlet and surcharge terms exchange water between the surface and network. The same timestep multiplies all contributions in an accepted update.

Table 3.1: Ordered production update and its conservative or diagnostic role.

Stage	Operation	Role in the update
1	Hydrostatic HLL surface flux	Transfers depth and momentum across mesh edges and applies balanced bed-pressure corrections
2	Manning friction and speed control	Damps momentum, protects dry cells and enforces the registered velocity ceiling
3	Rainfall and Horton infiltration	Adds gross precipitation and removes only the capacity- and availability-limited infiltration depth
4	External inflow and inlet exchange	Applies prescribed surface input and transfers available water between mapped surface cells and drainage nodes
5	Pipes, outfalls and surcharge	Advances network conveyance, discharges at enabled outfalls and returns excess storage to the surface
6	Recharge and diagnostics	Updates recharge storage and loss, wet time, extrema, cumulative ledgers and scheduled snapshots

The adaptive timestep is recomputed from the current hydraulic state and reused only under the configured safety policy.

3.9 System mass ledger

For any accepted time interval, the diagnostic balance can be written generically as

$$R_M = V_{s,0} + V_{n,0} + \sum V_{\text{in}} - \sum V_{\text{loss}} - V_{s,f} - V_{n,f} - V_{r,f}, \quad (3.13)$$

where V_s , V_n and V_r are surface, drainage-node and explicitly retained recharge storage. Inputs include rainfall and prescribed external inflow. Losses include infiltration, net outward boundary transfer, outfall discharge and recharge removal. Surface-to-network capture and surcharge are internal exchanges: they are recorded for diagnosis but cancel when the complete coupled system is considered. The solver convention records boundary net flux as positive inward and negative outward, so sign conversion is required when it is placed in the loss column of a presentation table.

The residual R_M measures numerical closure of the registered ledger; it is not an unmodelled physical discharge. Its ratio uses the package’s common comparison denominator. The JAX report provides both a CuPy-compatible residual, which omits final recharge storage to match the reference ledger, and a complete residual that includes it. This dual reporting preserves a like-for-like regression check without discarding the more complete accounting.

Chapter 4

GPU Implementation Architecture

4.1 One model, two accelerator paths

The comparison is designed around mathematical equivalence, not identical source code. Both paths load the same hashed arrays and preserve state ordering, units, single-precision surface variables, source sequence and output schedule. The state is represented by structure-of-arrays fields for h , hu , hv , wet time, node storage, link discharge and recharge storage. Edge-local work and indexed accumulation dominate the surface update; graph-local work dominates drainage exchange.

4.2 CuPy/CUDA implementation

CuPy exposes NumPy-compatible arrays on CUDA devices [7]. The reference backend uses fused custom CUDA production kernels for surface and drainage operations, reducing Python dispatch and avoiding host-device transfers during stepping. The recorded implementation identifier is `gurugram_flood_backend_fused_cuda`. Device arrays remain on CUDA device 0; only final products and diagnostics are transferred for storage. This backend serves as the numerical reference for parity, not as physical truth.

4.3 JAX/XLA implementation

JAX traces pure array functions and lowers them through XLA [8]. The production path uses immutable named state tuples, scatter-add reductions for edge and network accumulation, and a compiled `while_loop`. Inactive tail iterations are avoided. Compilation and execution are synchronized before timing. The first compiled output chunk includes compilation and execution, while subsequent chunk runtimes represent the already compiled path. The implementation identifier is `hybrid_flood_v2_jax_v1_parity`.

4.4 Role of PyTorch

PyTorch supports imperative tensor computation and CUDA acceleration [9]. It was available in the production environment and its CUDA capability was confirmed, but it was not an execution backend for the final complete-domain experiment. The implementations timed here are CuPy and JAX. Therefore, no PyTorch runtime is placed in the benchmark table and no CuPy-PyTorch or JAX-PyTorch speed claim is made.

4.5 Execution environment

Table 4.1: Recorded production GPU and software environment. *Source: completed-project evidence package.*

Item	Recorded value
GPU	NVIDIA RTX PRO 4000 Blackwell, 24,467 MiB
Driver / CuPy CUDA runtime	595.84 / 13.2
Python	3.12.14
CuPy / NumPy	14.1.1 / 2.5.1
JAX / jaxlib	0.10.2 / 0.10.2
PyTorch	2.13.0+cu130; CUDA availability confirmed
Selected devices	CuPy device 0; JAX <code>cuda:0</code>
Fallback	CPU fallback false
Primary surface precision	float32

The environment record is essential because compiler, driver and GPU architecture can affect both floating-point ordering and performance. The report therefore interprets the timings as results for this specific hardware–software configuration rather than universal framework rankings.

4.6 Parallel work and device-resident data

The surface update exposes one large parallel problem over 1,490,015 edges. Each edge reads the states and bed elevations of its adjacent triangles, constructs wet–dry-safe interface states, evaluates wave speeds and produces flux contributions. Those contributions must then be accumulated into as many as 981,880 cell states. The drainage calculation has a corresponding graph structure over 139,798 links and 118,768 nodes, followed by sparse surface–network exchanges at mapped inlets. These indexed accumulations, rather than the three-component state vector alone, are the principal implementation challenge.

Both implementations store major fields as separate contiguous arrays. This structure-of-arrays organisation lets adjacent GPU threads read the same physical quantity for different edges or cells and avoids repeatedly moving structured Python objects. Static geometry, terrain, coefficients and connectivity are uploaded once; evolving depth, momentum, wet time and network storage remain device resident through the timestep loop. Only scheduled results and scalar diagnostics cross the device boundary. For a single depth-history array alone, 25 float32 states over 981,880 cells represent 24,547,000 finite values, approximately 93.6 MiB before archive compression.

CuPy expresses the production-critical work through fused CUDA kernels and explicit device arrays. JAX represents the corresponding operations as traceable array functions, uses scatter reductions for indexed contributions and places the time loop inside a compiled XLA control-flow operation. These are different execution strategies for the same staged calculation. Numerical parity is consequently judged on physical outputs and ledgers, not on kernel count or identical floating-point instruction order.

4.7 Compilation, synchronization and output boundaries

GPU launches are asynchronous with respect to the host unless a result is explicitly synchronized. The recorded timing path therefore waits for accelerator completion before accepting a runtime. JAX compilation is also separated conceptually from steady execution: the first output chunk records compilation together with execution, and the remaining chunks use the compiled path.

Hourly output targets do not replace the adaptive solver clock. The solver advances with subsecond-to-one-second steps and stores the first state after one second followed by requested

hourly targets through 86,400 seconds. Recording this offset prevents spatial fields from being compared as though their timestamps were mathematically exact and identical.

Table 4.2: Backend correspondence for the same numerical responsibilities.

Responsibility	CuPy/CUDA path	JAX/XLA path	Required agreement
Surface state	device float32 arrays	immutable float32 array state	cell order, units and initial values
Edge accumulation	fused CUDA kernels	XLA scatter reductions	conservative contribution to each adjacent cell
Network state	device node/link arrays	compiled named state tuple	topology, capacities and exchange ordering
Time integration	host-controlled GPU kernels	compiled XLA while loop	CFL rule, maximum step and output targets
Acceptance	reference products and ledger	candidate products and ledger	all eight predefined parity gates

Chapter 5

Verification and Experimental Methodology

5.1 Evidence hierarchy

Three forms of evidence are kept separate. Software verification asks whether tests and invariants pass. Numerical verification asks whether two implementations of the same model agree within registered tolerances. Physical validation asks whether model predictions agree with observations. The first two are available; the third is not. ANUGA adds an independent numerical reference on compatible small cases but cannot replace observations.

5.2 Production experiment

Both backends ran the same 24 h event with 160.09 mm accumulated rainfall, dry surface initial conditions, topology-defined drainage storage, CFL 0.85, maximum timestep 1 s, maximum speed 5 m s^{-1} , float32 surface fields and hourly outputs. CuPy was treated as the reference implementation and JAX as the candidate. This designation defines the direction of errors; it does not imply that CuPy is an observation.

5.3 Parity metrics and acceptance gates

For valid cells $i = 1, \dots, N$, depth error is summarized by

$$\text{RMSE}_h = \sqrt{\frac{1}{N} \sum_i (h_i^J - h_i^C)^2}, \quad \text{MAE}_h = \frac{1}{N} \sum_i |h_i^J - h_i^C|. \quad (5.1)$$

With wet sets W_C and W_J defined at the registered threshold, flood-extent overlap is

$$\text{CSI} = \frac{|W_C \cap W_J|}{|W_C \cup W_J|}. \quad (5.2)$$

Relative surface-volume error, maximum-depth error, snapshot-time alignment and source–sink ledger differences complete the gates. The acceptance limits were fixed before interpreting the comparison; their observed values are reported later with the production parity results table 7.1.

5.4 Repeatability protocol

A second independent 24-hour run was compared with run A for each backend using the existing criteria: depth RMSE at most 0.05 m, minimum CSI at least 0.95, and snapshot offset at most 1 s. Archive identity was evaluated separately. This distinction prevents scientifically acceptable floating-point differences from being incorrectly reported as bitwise determinism.

5.5 Performance protocol

The completed repeated timing experiment is a secondary, identical 60-minute production workload. Each backend completed three warm-ups and ten synchronized measurements on the same device, and every measured output passed its correctness check. Mean, median, range, population standard deviation, interquartile range and coefficient of variation were computed from raw measurements. The longer official 24-hour repeated benchmark was stopped after 10 CuPy and 3 JAX measurements; it therefore provides no official backend ratio and is not merged with the complete secondary benchmark.

5.6 Controls against an invalid comparison

The comparison was constrained at four levels. At the input level, both implementations loaded the same topology, terrain, roughness, infiltration, rainfall, recharge, drainage and boundary assets; manifests and hashes preserve their identity. At the model level, the state variables, physical constants, wet threshold, source ordering, CFL coefficient, maximum timestep and output targets were matched. At the execution level, both backends selected `cuda:0`, recorded GPU status and prohibited CPU fallback. At the evaluation level, the parity limits existed before the final interpretation and were applied to every hourly state rather than only the final map.

CuPy was called the numerical reference because the JAX path was designed to reproduce its production sequence. This is a software-comparison convention, not an assumption that CuPy is exact. Errors are consequently signed or labelled as JAX relative to CuPy, while physical accuracy remains unassessed. The same distinction applies to mass balance: the residual regression gate detects whether JAX materially worsens the compatible CuPy ledger, whereas the complete JAX ledger separately accounts for ending recharge storage.

5.7 Software verification and package audit

The final software check collected 148 tests. Of these, 111 passed, 37 were explicitly skipped, and none failed or raised an error. The run completed in 10.58 s with two warnings. Skipped tests were classified as outside the final production scope or as requiring the opposite hardware environment; they were not counted as passes. The result therefore reports both the successful executable baseline and the exact portion that was not exercised.

Table 5.1: Final automated-test outcome and interpretation.

Outcome	Count	Interpretation
Collected	148	Total discovered by the test runner
Passed	111	Executed successfully
Explicitly skipped	37	Excluded-scope or opposite-hardware classification
Failed / errors	0 / 0	No failing test or collection/runtime error
Warnings / runtime	2 / 10.58 s	Preserved runner diagnostics

The semantic package audit checked that required reports, machine-readable tables, simulation archives, figures, environment records and reproduction material were present and mutually consistent. SHA-256 verification passed. These controls establish provenance: they do not improve a numerical score, but they identify the evidence supporting each reported result.

Chapter 6

Production Simulation Results

6.1 Surface and network response

The two 24-hour runs produced closely aligned domain-scale responses. CuPy used 321,645 timesteps and JAX used 321,640. The domain-wide maximum depths were 9.414836884 m and 9.414829254 m, respectively, while the final numbers of cells deeper than 0.05 m were 129,187 and 129,180. The extreme maximum is a local model value and should not be interpreted as a typical city depth or as a measured water level.

Table 6.1: Principal 24-hour production outputs.

Quantity	CuPy	JAX
Solver timesteps	321,645	321,640
Recorded solver runtime (s)	430.467	451.865
Maximum depth (m)	9.414836884	9.414829254
Wet triangles at 0.05 m	129,187	129,180
Final surface volume (m ³)	23,377,569.249	23,375,823.126
Final node volume (m ³)	420,425.063	420,421.469
Rainfall input (m ³)	47,701,360.545	47,701,532.000
Infiltration loss (m ³)	4,545,656.989	4,545,672.500
Outfall discharge (m ³)	1,060,887.868	1,060,836.750

6.2 Preserved state and diagnostic products

The production archives retain more than a final depth map. Each backend stores depth and speed histories with shape $25 \times 981,880$, final momentum components, event maxima, surface masks, terrain, coordinates and mesh connectivity. Network products include a $25 \times 118,768$ node-volume history and final discharge for all 139,798 links. Every entry of the principal history arrays is finite. This output design supports temporal, spatial and conservation checks from the same run rather than requiring the solver to be executed again for each figure.

Table 6.2: Selected preserved arrays from the CuPy production archive; the JAX archive has corresponding fields.

Array	Shape	Units	Recorded range or role
Depth snapshots	$25 \times 981,880$	m	24,547,000 finite values; event maximum 9.414837 m
Speed snapshots	$25 \times 981,880$	m s^{-1}	event maximum 1.712713 m s^{-1}
Drainage-node volume snapshots	$25 \times 118,768$	m^3	temporal network storage for every node
Final h , hu , hv	981,880 each	m; $\text{m}^2 \text{s}^{-1}$	complete final conserved surface state
Final link flow	139,798	$\text{m}^3 \text{s}^{-1}$	-1.56136 to $3.40805 \text{ m}^3 \text{s}^{-1}$
Geometry and masks	mesh-dependent	m / indices	coordinates, triangles, terrain, active, road and building context

The tiny CuPy minimum depth in the full history, -8.73×10^{-11} m, is far below the registered -10^{-7} m non-negativity tolerance and the final minimum is approximately -5.68×10^{-14} m. JAX stored non-negative depths. These values are retained rather than rounded away because they document float32 numerical behaviour at nominally dry cells. They do not represent physically negative water depth.

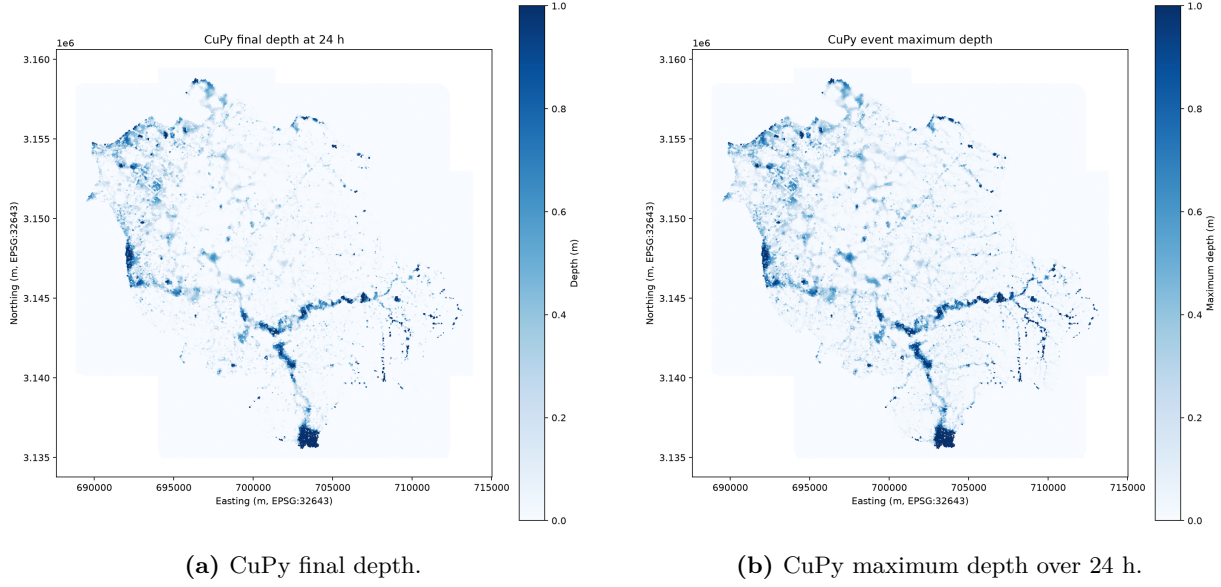


Figure 6.1: Production surface-water results. These maps are model outputs and have not been validated against observed inundation.

6.3 Wet extent and timing

The wet-area threshold for production summaries is 0.05 m. Final wet-cell counts differed by seven triangles, consistent with the final CSI of 0.999590. Arrival and duration products were derived from the 25 stored states and therefore have hourly temporal resolution; they must not be interpreted as minute-scale timing observations.

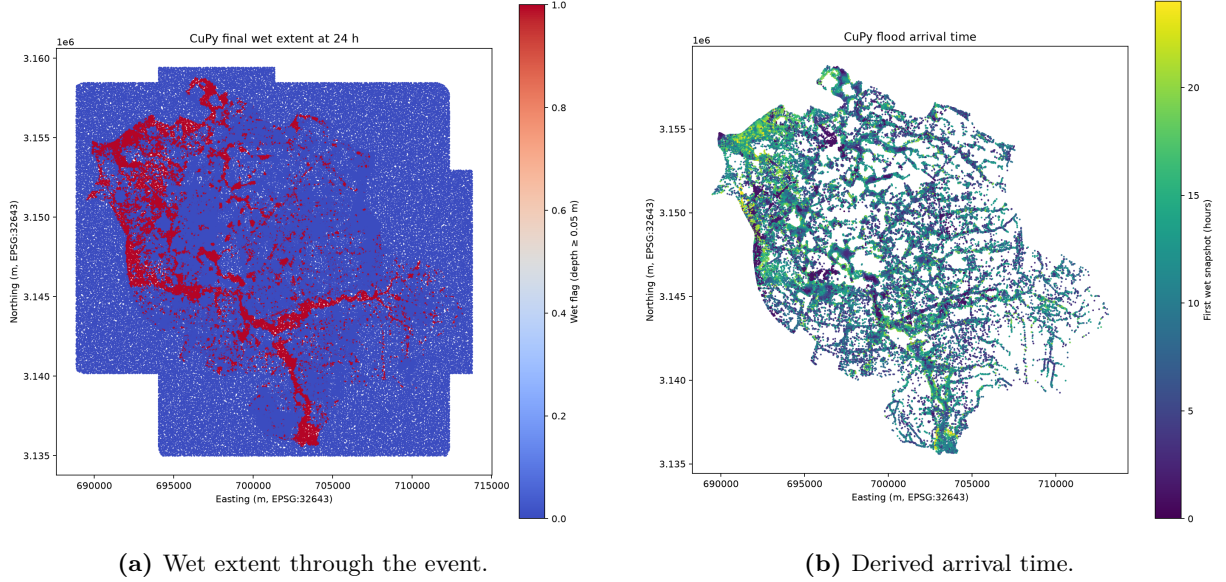


Figure 6.2: Thresholded flood extent and arrival-time products.

6.4 Flood duration and velocity diagnostics

Flood duration counts the stored hourly states in which each cell exceeds the registered 0.05 m wet threshold. Because the first stored state occurs after one second and the remaining targets are hourly, the maximum count is 25; the product is a count of wet snapshots, not a continuously integrated duration. Long-duration cells form connected and localized corridors within the active domain, whereas most triangles never cross the threshold. This pattern is consistent with the final

wet count being much smaller than the 817,573 active cells.

The event maximum-speed product is calculated from the depth-averaged velocity history. In the CuPy archive its domain-wide maximum is $1.712713 \text{ m s}^{-1}$, its all-cell mean is $0.0421213 \text{ m s}^{-1}$, and all 981,880 entries are finite. For JAX, the corresponding maximum and mean are 1.712712 and $0.0421171 \text{ m s}^{-1}$. Both maxima remain below the model’s registered 5 m s^{-1} safety ceiling, so the reported event maximum is not equal to the cap.

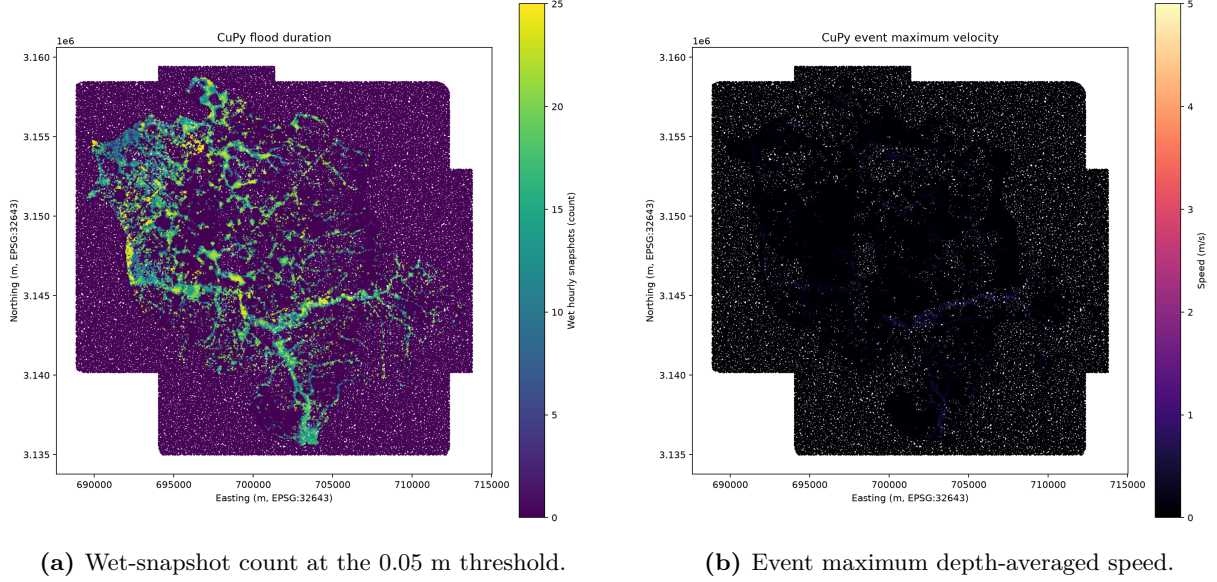


Figure 6.3: Derived CuPy duration and maximum-velocity products from the preserved 24-hour histories.

The velocity map should be interpreted as a hydraulic-model diagnostic rather than a pedestrian or vehicle hazard classification. No external hazard standard was imposed in the completed experiment, and velocity alone omits depth and exposure. Likewise, a wet-snapshot count supports comparison within this output schedule but cannot resolve sub-hourly arrival or recession between saved states.

6.5 Mass-balance accounting

The CuPy compatible ledger residual was $136,495.864 \text{ m}^3$, or 0.269684% of its registered denominator. The JAX compatible residual was $147,573.342 \text{ m}^3$, or 0.291569%. JAX additionally tracks ending recharge storage explicitly; including $63,394.368 \text{ m}^3$ gives a complete JAX residual of $84,178.974 \text{ m}^3$, or 0.166317%. The parity gate is a regression criterion: the candidate compatible residual ratio may exceed the CuPy ratio by no more than 0.001. It is not an assertion that absolute residual must be below 0.1%. The observed increase was 0.000218855 and passed.

The ledger includes rainfall, infiltration, external inflow, boundary flux, inlet capture, surcharge, outfalls, recharge and ending surface/network storage. This explicit accounting makes it possible to locate imbalance without treating the final surface volume as the entire system mass.

Table 6.3: Detailed cumulative mass-ledger terms. Signs in the source reports follow their native conventions; magnitudes are shown for named losses and exchanges.

Ledger term (m ³)	CuPy	JAX
Initial drainage-node storage	2,128,865.685	2,128,865.750
Rainfall input	47,701,360.545	47,701,532.000
External boundary inflow	783,082.540	783,082.500
Net outward boundary transfer	21,060,985.221	21,051,882.000
Infiltration loss	4,545,656.989	4,545,672.500
Outfall discharge	1,060,887.868	1,060,836.750
Overflow / surcharge diagnostic	16,184,457.202	16,182,797.000
Final surface storage	23,377,569.249	23,375,823.126
Final drainage-node storage	420,425.063	420,421.469
Final recharge storage	not explicit in compatible ledger	63,394.368

Rainfall is the largest recorded external input, while net boundary transfer is the largest named outward term. The overflow/surcharge values describe internal network-to-surface exchange and must not be added as a new external source when closing the whole-system balance. At 24 hours, surface storage is approximately 23.38×10^6 m³ and node storage approximately 0.420×10^6 m³ in both implementations.

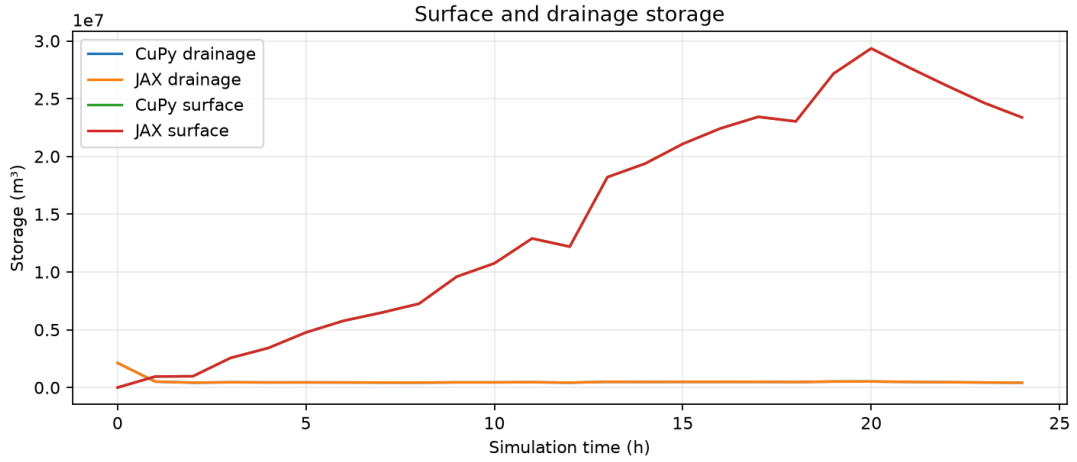


Figure 6.4: Surface and drainage-node storage through the production event. The nearly overlapping backend curves provide a time-resolved view of the parity summarized by scalar gates.

Surface storage rises in steps that follow the temporally varying rainfall and other source terms, reaches its largest plotted value near hour 20, and then declines toward the final state. Drainage storage begins from the topology-defined nonzero initial condition, changes rapidly during the first hour, and remains much smaller than surface storage on the common vertical scale. The CuPy and JAX curves visually overlap throughout; the numerical parity chapter quantifies the remaining differences rather than relying on visual agreement.

Chapter 7

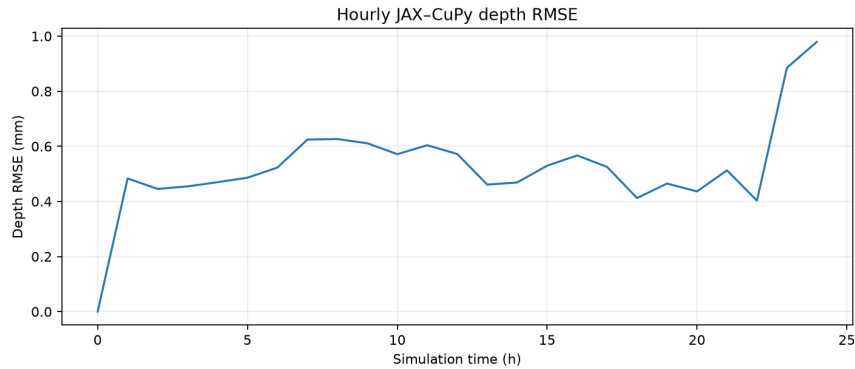
Numerical Parity and Repeatability

7.1 Cross-backend parity

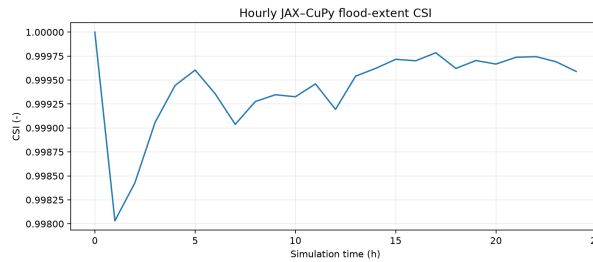
All eight registered gates passed. The largest hourly depth RMSE was only 1.96% of the 0.05 m limit, and the minimum CSI exceeded its 0.95 limit by 0.048. The final surface-volume difference was approximately $-1,746.12 \text{ m}^3$, corresponding to -0.00747% , while the maximum absolute hourly volume error was 0.03329%.

Table 7.1: Approved parity gates and observed values.

Gate	Approved limit	Observed	Result
Minimum hourly flood-extent CSI	≥ 0.95	0.998031	Pass
Maximum hourly depth RMSE	$\leq 0.05 \text{ m}$	0.000979348 m	Pass
Maximum hourly volume error	$\leq 2\%$	0.033294%	Pass
Finite/non-negative depth	$\text{min} \geq -10^{-7} \text{ m}$	finite/non-negative	Pass
Residual-ratio regression	increase ≤ 0.001	0.000218855	Pass
Maximum-depth relative error	$\leq 5\%$	0.0000810%	Pass
Snapshot-time mismatch	$\leq 1 \text{ s}$	0.318359 s	Pass
Source-sink ledger difference	$\leq 2\%$	1.543653%	Pass



(a) Hourly depth RMSE.



(b) Hourly flood-extent CSI.

Figure 7.1: Cross-backend parity through the 24-hour event.

7.2 Spatial differences

At the final state, depth RMSE was 0.000979348 m, CSI was 0.999590 and the domain-wide maximum-depth values differed by $7.63 \times 10^{-6} \text{ m}$. Local differences are spatially heterogeneous and may be amplified near wet-dry transitions even when global errors remain small. The difference map is therefore more informative than a single scalar alone.

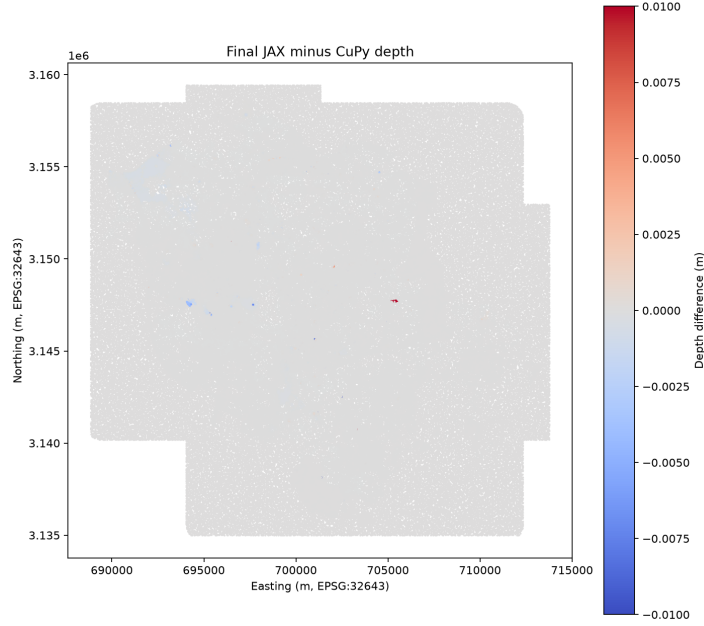


Figure 7.2: Spatial difference between final JAX and CuPy depths. CuPy is the implementation reference, not an observed truth field.

7.3 Within-backend repeatability

Independent run-B executions passed the pre-existing repeatability criteria for both backends. CuPy produced RMSE 0.000416766 m, minimum CSI 0.998740 and maximum snapshot offset 0.354492 s; its timestep count differed by 28. JAX produced RMSE 0.000406100 m, minimum CSI 0.998031 and maximum snapshot offset 0.337891 s; its timestep count differed by 24.

Table 7.2: Independent 24-hour within-backend repeatability.

Backend	RMSE (m)	Minimum CSI	Offset (s)	Δ steps	Bitwise
CuPy	0.000416766	0.998740	0.354492	28	No
JAX	0.000406100	0.998031	0.337891	24	No

Neither output archive was byte-identical. The evidence shows association with adaptive timestep and snapshot-time divergence, but it does not isolate a lower-level cause among GPU reduction order, floating-point non-associativity and wet–dry sensitivity. The correct classification is therefore *numerically repeatable within the approved criteria, but not bitwise deterministic*.

7.4 Interpretation of localized differences

Global norms and spatial extremes answer different questions. The repeat-run maximum absolute depth differences reached 0.283989 m for CuPy and 0.603012 m for JAX, even though their domain-wide RMSE values remained below 0.000417 m. This combination indicates that the largest changes are localized rather than representative of the entire mesh. Reporting only RMSE would conceal those outliers; reporting only the maximum would incorrectly imply a domain-wide discrepancy. The simultaneous CSI values above 0.998 show that thresholded inundation support remained very similar.

Wet–dry transitions are a plausible location for amplified local differences because a small depth change can alter whether momentum is retained, whether an inlet has water available, and which adaptive timestep is selected.

The two repeats also had different step counts and slightly different snapshot times. These observations establish association, not a proven low-level cause. Isolating the cause would require controlled kernel-level tests of reduction order, timestep selection and threshold crossings, which were not part of the completed experiment.

The parity conclusion therefore rests on multiple complementary metrics: RMSE and MAE for distributed depth error, CSI for flood support, volume and ledger terms for conservation, maximum-depth comparison for the event extreme, and time alignment for snapshot validity. Agreement across these metrics justifies numerical equivalence within the approved gates, while the lack of byte identity is retained as a separate and transparent result.

Table 7.3: Selected time-resolved observations from the complete 25-state parity table.

Time (h)	Depth MAE (m)	Depth RMSE (m)	Flood CSI	ΔV_s (m ³)
0*	4.6920×10^{-10}	3.9743×10^{-8}	1.000000	−0.0046
1	8.8698×10^{-6}	4.8343×10^{-4}	0.998031	104.395
6	1.5106×10^{-5}	5.2301×10^{-4}	0.999357	−940.855
12	2.7740×10^{-5}	5.7215×10^{-4}	0.999194	−3545.801
18	3.1494×10^{-5}	4.1220×10^{-4}	0.999622	−4509.684
24	5.2020×10^{-5}	9.7935×10^{-4}	0.999590	−1746.123

*The first stored state is at 1 s and is labelled as the initial output in the package. $\Delta V_s = V_{s,JAX} - V_{s,CuPy}$.

The selected rows illustrate why acceptance was evaluated over the entire event. The minimum CSI occurred near the beginning rather than at the final output, whereas the largest depth RMSE occurred at 24 hours. Surface-volume difference also changed sign and magnitude through time. A final-state-only comparison would therefore miss the worst extent agreement and would provide no evidence that the intermediate trajectories remained close.

Table 7.4: Why no single metric is sufficient for the completed numerical-equivalence decision.

Metric family	Question answered	Failure it can reveal
MAE, RMSE and relative L_2	How different are distributed depth fields?	Broad bias or accumulated field disagreement
CSI, precision and recall	Do the backends inundate the same cells?	Shifted wet–dry fronts hidden by a small domain-average error
Surface volume and ledger	Are integrated water quantities consistent?	Source, sink or exchange accounting mismatch
Maximum depth	Is the event extreme reproduced?	Local peak amplification or suppression
Snapshot-time offset	Are fields compared at equivalent times?	Apparent spatial error caused by adaptive-clock misalignment

Chapter 8

GPU Performance Comparison

8.1 Completed secondary benchmark

The complete repeated comparison used a 60-minute production workload, three warm-ups and ten accepted measurements for each backend. All twenty measured results passed correctness checks. CuPy median solver runtime was 17.267041 s; JAX median runtime was 18.212524 s. With

$$\Delta_{J/C} = 100 \left(\frac{T_J}{T_C} - 1 \right), \quad (8.1)$$

the measured difference is 5.475648%, meaning JAX was slower for this specific workload, device and software environment.

Table 8.1: Secondary 60-minute production benchmark.

Statistic	CuPy	JAX
Warm-ups / measurements	3 / 10	3 / 10
Mean runtime (s)	17.333616	18.215309
Median runtime (s)	17.267041	18.212524
Minimum–maximum (s)	16.983008–17.854433	18.185589–18.251359
Population standard deviation (s)	0.333464	0.021280
Interquartile range (s)	0.628316	0.025221
Coefficient of variation	0.019238	0.001168
Median steps per second	758.443	718.515
Simulated h per wall-clock h	208.507	197.666

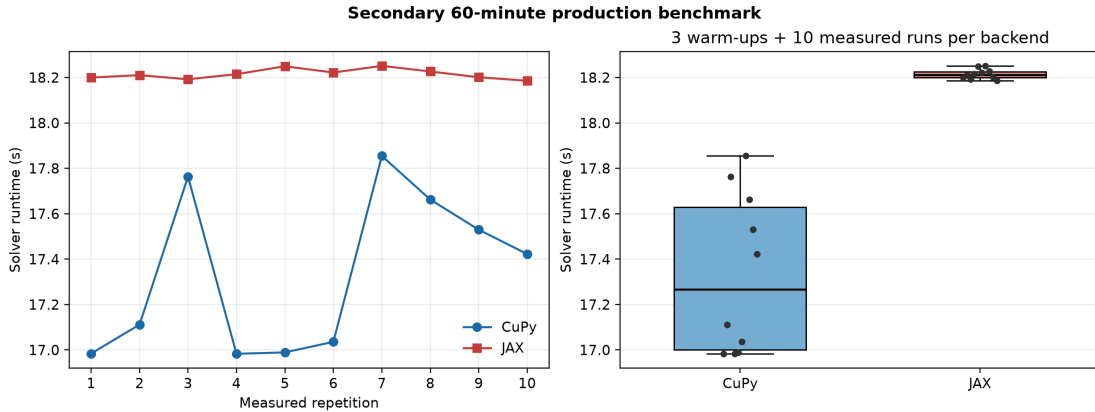


Figure 8.1: Measured solver runtimes for ten repetitions per backend after three warm-ups. The figure was generated from the verified `benchmarks/secondary_60min/benchmark_results.csv` records.

8.2 Interpretation

The result does not imply that CuPy is universally faster than JAX. CuPy benefits here from production-specific fused CUDA kernels, while JAX provides compiler-managed fusion, functional transformations and a path toward differentiable or ML-coupled workflows. The CuPy timings show greater run-to-run dispersion, whereas the JAX timings are tightly clustered, but ten measurements are insufficient for broad hardware-general conclusions. The benchmark supports only the claim that CuPy had the lower median solver runtime in the completed secondary experiment.

8.3 Unfinished official 24-hour benchmark

The official protocol required three warm-ups and ten 24-hour measurements per backend. CuPy completed ten measurements (median 426.089 s); JAX completed only three (partial median 448.328 s) before the stop. The JAX sample is incomplete, so no official 24-hour ratio, confidence interval or significance test is reported. The single timings in table 6.1 also remain descriptive rather than benchmark results. These partial counts and timings are retained in the supplementary interrupted-run progress record, separate from the completed secondary-benchmark evidence.

Chapter 9

ANUGA Numerical Reference

ANUGA is an external finite-volume shallow-water model and is neither the CuPy nor JAX implementation [10, 11]. Version 3.3.10 was run independently on compatible two-triangle controlled cases. Still-water and rainfall-only comparisons passed the existing absolute mass criterion of $5 \times 10^{-8} \text{ m}^3$. The moving-water dam-break comparison is descriptive because reconstruction and numerical flux choices differ.

Table 9.1: Available controlled-case ANUGA–JAX evidence. *Source: ANUGA controlled-case evidence.*

Case	Depth RMSE (m)	Volume difference (m ³)	Classification
Still water	2.980232×10^{-9}	2.980232×10^{-9}	Criterion passed
Rainfall only	5.518733×10^{-12}	5.518733×10^{-12}	Criterion passed
Moving-water dam break	6.398733×10^{-3}	-3.725290×10^{-9}	Descriptive only
Terrain lake at rest	—	—	ANUGA-only diagnostic; zero stage drift

A full production-domain cross-model run was deliberately not executed. The current ANUGA adapter cannot identically represent the production drainage, recharge, surcharge, outfall, external-hydrograph and outflow-only boundary system. Removing those components would form a different model and would not be a valid production comparison. The available ANUGA work supports limited implementation verification on compatible physics; it is not validation against real flood observations.

Chapter 10

Limitations and Future Work

The strongest limitation is the absence of compatible observations. Without surveyed water levels, flood outlines, discharge hydrographs or arrival times, the model cannot be assigned physical predictive accuracy. The DEM vertical datum is undocumented, which limits absolute water-level interpretation. Only one production rainfall event is available, so the results do not quantify event-to-event generalization and cannot support leakage-controlled production AI splits.

The official repeated 24-hour benchmark is incomplete for JAX. The complete 60-minute benchmark is valid but secondary and hardware-specific. Numerical repeatability was achieved without byte-level identity, and the low-level origin of the small divergence was not isolated. The ANUGA reference is restricted to small compatible cases rather than the full production drainage model.

Future work should prioritize observed flood data with documented spatial and temporal support, DEM vertical-datum confirmation, and independent rainfall events. Calibration and validation must use separate events. The remaining seven JAX measurements should then be completed before publishing an official 24-hour timing comparison. Multiple GPUs and software versions would establish performance portability. If enough independent events become available, complete-event train/validation/test partitions can support mesh-aware AI forecasting without temporal leakage. Any surrogate should be evaluated against persistence and physics-only baselines and should preserve uncertainty and failure reporting.

Chapter 11

Conclusions

This project implemented the same production urban flood model through CuPy/CUDA and JAX/XLA on a single GPU. The executed model includes the complete triangular surface, terrain, spatial roughness, rainfall, Horton infiltration, recharge, dynamic drainage, surcharge, outfalls, external inflow and boundary treatment.

The central numerical objective was achieved: all eight cross-backend parity gates passed. The final depth RMSE was 0.000979348 m, final flood-extent CSI was 0.999590 and the domain-wide maximum depths agreed to approximately 7.63×10^{-6} m. Both backends also passed independent 24-hour numerical repeatability criteria, although neither was bitwise deterministic. The software baseline passed 111 tests, with 37 explicitly classified skips and no failures or errors.

The complete secondary repeated benchmark showed CuPy median runtime 17.267 s and JAX median runtime 18.213 s; JAX was 5.476% slower on the tested 60-minute workload. This is the defensible performance result. The incomplete 24-hour repeated benchmark is reported only as unfinished evidence. Controlled ANUGA cases provide limited independent numerical support but not production physical validation.

Accordingly, the project is complete as a GPU numerical implementation, verification, repeatability and secondary performance study. It is not complete as an observationally validated forecasting system or production AI study. This qualified conclusion is more scientifically precise than treating successful computation as proof of real-world accuracy.

Reproducibility and Evidence Statement

The authoritative source for the completed production runs, parity, repeatability, software verification and secondary benchmark is the repository folder `HF_CUPY_JAX_COMPLETED_PROJECT`. The project repository is publicly available at <https://github.com/arsanjeevk/gpu-urban-flood-cupy-jax> [1]. Key evidence consists of `COMPLETION_ADDENDUM.md`, test output and summary, backend run reports, hourly parity tables, validation gates, mass ledgers, repeatability metrics, raw secondary benchmark measurements, environment metadata, source manifests and checksums. Supplementary ANUGA controlled-case evidence is retained in `Ravi-20260903T125449Z-1-001/Ravi/evidence`; the interrupted official 24-hour benchmark counts and partial timings are retained in that package’s `CURRENT_PROGRESS_REPORT.md`. Neither supplementary record overrides the completed-project results.

The packaged archive SHA-256 checksum is

d9b4656055b728de582be8d5fb53892cff659d7ef5a5f29312365bb15bb7476c.

Table 11.1: Evidence groups used to construct and independently check the report.

Evidence group	Preserved material
Inputs and provenance	Production configurations, rainfall series, source manifest and asset hashes
Simulation state	CuPy and JAX NPZ archives, run reports and array catalogue
Numerical verification	Hourly parity table, eight validation gates, spatial-difference arrays, mass ledgers and repeatability metrics
Performance	Raw secondary measurements, summary statistics and benchmark protocol
Reproduction	Software/GPU environment, test output, commands, scripts and semantic package audit

The report PDF is a synthesis of existing evidence: no solver or simulation output was modified or regenerated during report preparation.

Bibliography

- [1] S. Kumawat, *GPU-Accelerated Physics-Based Urban Flood Modelling Using CuPy/CUDA and JAX/XLA*, project source code, configurations, verification evidence and report sources, 2026. Available: <https://github.com/arsanjeevk/gpu-urban-flood-cupy-jax>.
- [2] R. J. LeVeque, *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press, 2002. doi:10.1017/CBO9780511791253.
- [3] A. Harten, P. D. Lax, and B. van Leer, “On upstream differencing and Godunov-type schemes for hyperbolic conservation laws,” *SIAM Review*, vol. 25, no. 1, pp. 35–61, 1983. doi:10.1137/1025002.
- [4] E. Audusse, F. Bouchut, M.-O. Bristeau, R. Klein, and B. Perthame, “A fast and stable well-balanced scheme with hydrostatic reconstruction for shallow water flows,” *SIAM Journal on Scientific Computing*, vol. 25, no. 6, pp. 2050–2065, 2004. doi:10.1137/S1064827503431090.
- [5] R. E. Horton, “The role of infiltration in the hydrologic cycle,” *Transactions, American Geophysical Union*, vol. 14, no. 1, pp. 446–460, 1933. doi:10.1029/TR014i001p00446.
- [6] V. T. Chow, *Open-Channel Hydraulics*. New York, NY, USA: McGraw-Hill, 1959.
- [7] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Workshop on Machine Learning Systems, NeurIPS*, 2017.
- [8] R. Frostig, M. J. Johnson, and C. Leary, “Compiling machine learning programs via high-level tracing,” in *Systems for Machine Learning*, 2018.
- [9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, pp. 8024–8035, 2019.
- [10] S. Roberts, G. Davies, O. Nielsen, et al., *ANUGA Hydrodynamic Inundation Modelling*. Australian National University and Geoscience Australia. Available: https://github.com/anuga-community/anuga_core.
- [11] C. Zoppou and S. Roberts, “Catastrophic collapse of water supply reservoirs in urban areas,” *Journal of Hydraulic Engineering*, vol. 125, no. 7, pp. 686–695, 1999. doi:10.1061/(ASCE)0733-9429(1999)125:7(686).